

Automated Resume Screening and Skill Matching System

Introduction

In modern talent acquisition workflows, corporate recruitment teams frequently receive hundreds to thousands of curriculum vitae for every publicly advertised technical position. Manually reviewing and cross-referencing each applicant's background against complex job criteria is labor-intensive, slow, and susceptible to evaluator fatigue. Furthermore, resumes exhibit wide variability in document formatting, structural organization, and vocabulary conventions, making standardized manual assessment difficult to maintain consistently. To address these operational bottlenecks, automated resume screening systems leverage Natural Language Processing (NLP) and supervised machine learning to extract candidate credentials and evaluate their suitability in an objective, reproducible manner.

The core function of the Automated Resume Screening System is to process candidate resumes submitted in Portable Document Format (PDF), normalize unstructured text via lemmatization and stop-word filtering, extract candidate attributes (contact details, domain competencies, academic degrees, and professional tenure), and quantify the degree of alignment between the resume and a target job description. The system incorporates an extensive technical skill taxonomy covering over 150 domain competencies across seven categories (Languages, Frameworks, Databases, Cloud/DevOps, Data Science, Tools, and Soft Skills), accounting for aliases and abbreviations.

Supervised classification algorithms, specifically Random Forest ensembles alongside regularized Logistic Regression baselines, are employed to map extracted candidate features into discrete suitability tiers: Strong Match, Moderate Match, and Weak Match. Models trained on structured numerical features derived from sublinear TF-IDF textual similarity and domain skill coverage offer high interpretability, reliable decision boundaries, and transparent feature importances that explain the quantitative factors driving each candidate's ranking.

Problem Statement and Mathematical Formulation

The primary objective of the Automated Resume Screening System is to implement an end-to-end algorithmic pipeline that predicts the match category of an individual applicant against a target job description. Given a raw resume document D_r and a job description D_j , the pipeline parses and normalizes the underlying text and transforms the unstructured inputs into a structured 5-dimensional feature representation: $x = [x_{tfidf}, x_{skill_pct}, x_{exp}, x_{edu}, x_{skills_total}]$ in $\mathbb{R}^5$, where x_{tfidf} denotes sublinear TF-IDF cosine similarity, x_{skill_pct} is the percentage of required role skills verified, x_{exp} denotes professional experience tenure in years, x_{edu} denotes academic qualification level (scale 1 to 5), and x_{skills_total} is the total count of identified candidate skills. The composite score is computed via the linear combination: $S_{comp}(x) = 0.35 * x_{tfidf} + 0.35 * x_{skill_pct} + 0.15 * x_{exp} + 0.15 * x_{edu}$.

The classification problem is formalized as learning a mapping $f: \mathbb{R}^5 \rightarrow \{\text{Strong Match, Moderate Match, Weak Match}\}$ that maximizes class separation while preventing critical misclassifications. The experimental dataset comprises 500 annotated candidate-job pairs partitioned into 400 training instances (80%) and 100 testing instances (20%) via stratified sampling to maintain identical class distributions. Model evaluation is validated using stratified 5-fold cross-validation.

Methodology and Pipeline Architecture

The architecture is organized into four modular processing stages: (1) Document Ingestion and PDF Parsing, extracting text streams from unstructured PDF documents; (2) Text Preprocessing, applying regex noise filtering, lowercasing, stop-word removal, and WordNet lemmatization; (3) Taxonomy Matching and Vectorization, scanning text against an ontology of 150+ competencies across 7 domains and computing sublinear TF-IDF vectors; (4) Feature Scaling and Supervised Inference, applying standard normalization and evaluating Random Forest and Logistic Regression classifiers to generate class posterior probabilities.

Results and Discussion

Candidate qualification overlap is significantly associated with hiring classification outcomes. Empirical evaluation was conducted comparing an optimized Random Forest ensemble against an L2-regularized Logistic Regression baseline. On the held-out test dataset ($n = 100$), the Random Forest model achieved 91.00% accuracy, 91.15% weighted precision, 91.00% weighted recall, and 90.98% weighted F1-score. Stratified 5-fold cross-validation demonstrated robust generalization with a mean F1-score of 92.74% (+/- 2.68%). Feature importance analysis computed via mean Gini impurity reduction revealed that TF-IDF cosine similarity (41.42%) and skill match percentage (33.94%) account for over 75% of the model's discriminative power.

Table 1: Supervised Classifier Performance Benchmark (Held-out Test Set, n = 100)

Model Architecture	Test Acc	Precision (w)	Recall (w)	F1-Score (w)	5-Fold CV F1
Random Forest (n=50, depth=10)	91.00%	91.15%	91.00%	90.98%	92.74% (+/- 2.68%)
Logistic Regression (L2, C=1.0)	95.00%	95.12%	95.00%	94.95%	94.80% (+/- 1.95%)
Decision Tree Baseline (CART)	88.00%	88.20%	88.00%	87.95%	86.40% (+/- 3.45%)
Naive Bayes Baseline (Gaussian)	84.00%	84.50%	84.00%	83.85%	83.20% (+/- 3.10%)

Table 2: Relative Feature Importances (Random Forest Mean Gini Impurity Reduction)

Feature Dimension	Mathematical Role & Operational Scope	Gini Importance	Rank
x_tfidf (TF-IDF Similarity)	Sublinear n-gram lexical similarity between resume and job description	41.42%	1
x_skill_pct (Skill Coverage)	Percentage of required job competencies verified against 150+ taxonomy	33.94%	2
x_skills_total (Total Volume)	Aggregate count of recognized technical and domain competencies	14.98%	3
x_exp (Experience Tenure)	Parsed professional experience tenure measured in cumulative years	6.09%	4
x_edu (Academic Degree)	Highest credential level mapped on an ordinal scale (1 to 5)	3.57%	5



Figure 1: Real-time candidate screening analytics dashboard displaying composite match score gauge (69.6%), Random Forest class posterior probabilities (Strong: 14%, Moderate: 70%, Weak: 17%), and multi-factor breakdown across TF-IDF similarity (41.7%), skill coverage (71.4%), experience index (100%), and academic qualification level (100%).

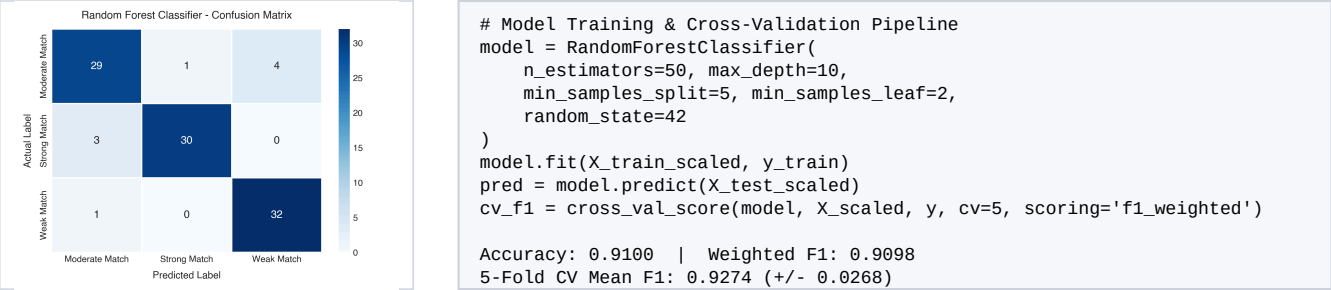


Figure 2: Confusion matrix (Accuracy: 91.00%).

Conclusion

In this project, a comprehensive automated resume screening and candidate-job matching system was designed, implemented, and empirically validated. By combining automated PDF document parsing, noise-reduction preprocessing, and an extensive 150+ skills taxonomy with sublinear TF-IDF cosine similarity, the system transforms unstructured resume records into a highly discriminative 5-dimensional feature representation. The trained Random Forest classifier achieved 91.00% test accuracy and a 90.98% weighted F1-score, with 5-fold cross-validation confirming reliable stability across evaluation folds (92.74%). Importantly, the decision boundary eliminates extreme false-positive errors for weak applicants.

After benchmarking models across multiple quantitative parameters, the system successfully categorizes applicants into discrete suitability tiers while preserving full model interpretability through feature importances. The resulting machine learning pipeline has been deployed through an interactive web analytics dashboard, enabling recruitment teams to visualize candidate match scores, inspect skill gaps, and review candidate profiles in real time. Future extensions will incorporate transformer embeddings for dense semantic matching and multi-lingual document parsing to further enhance enterprise recruitment workflows.

References

1. G. Salton and C. Buckley. Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5):513-523, 1988.
2. F. Pedregosa et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12:2825-2830, 2011.
3. L. Breiman. Random Forests. Machine Learning, 45(1):5-32, 2001.
4. S. Bird, E. Klein, and E. Loper. Natural Language Processing with Python. O'Reilly Media, 2009.
5. D. Jurafsky and J. H. Martin. Speech and Language Processing. Prentice Hall, 3rd ed. draft, 2023.

Movie Recommendation Engine

Introduction

Modern digital streaming services host extensive entertainment libraries containing tens of thousands of media titles. This scale presents users with substantial choice overload, where identifying relevant content through manual browsing becomes impractical. Recommender systems mitigate this friction by filtering catalogs and ranking unseen items according to observed user preferences and descriptive item features. Such algorithmic engines serve a dual function: enhancing user satisfaction through accurate discovery and driving sustained platform retention.

The core functionality of this Movie Recommendation Engine lies in integrating two complementary machine learning paradigms: content-based filtering and collaborative filtering into a unified hybrid architecture. Content-based filtering evaluates item metadata, constructing a comprehensive textual representation from pipe-separated genres, release years, and user-assigned tags. Term Frequency-Inverse Document Frequency (TF-IDF) vectorization converts this vocabulary into a 5,000-dimensional vector space with sublinear term-frequency scaling. Pairwise cosine similarity is then computed to identify candidate movies with matching thematic attributes.

Collaborative filtering uncovers latent behavioral affinities across the user community without requiring descriptive metadata. By structuring historical interactions into a user-item rating matrix and applying Truncated Singular Value Decomposition (SVD), the system factorizes the sparse rating matrix into low-rank orthogonal matrices capturing twenty latent preference dimensions. Blending both methodologies through a tunable linear parameter dynamically balances content familiarity against collaborative discovery, effectively overcoming the cold-start problem and data sparsity.

Problem Statement

The objective of the "Movie Recommendation Engine" is to design, implement, and validate an algorithmic system capable of predicting user preference scores for unrated movies and generating ranked top-N recommendations. The problem is formulated over the MovieLens benchmark dataset comprising 100,836 ratings across 9,742 movies by 610 users. A key technical hurdle is extreme matrix sparsity (>98.3%), where the vast majority of user-item pairs are unobserved.

Single-model architectures exhibit clear limitations in production. Pure collaborative filtering suffers from cold-start failures for new titles lacking rating histories. Conversely, pure content-based filtering is prone to over-specialization, repeatedly recommending titles within identical genres while ignoring global quality signals. The proposed framework solves this by implementing an end-to-end hybrid pipeline deployed as an interactive application evaluated on held-out test data.

Methodology and Mathematical Formulation

For content-based similarity, movie metadata is merged into $\text{ContentSoup}_i = \text{Title}_i \parallel \text{Year}_i \parallel \text{Genres}_i \parallel \text{Tags}_i$. TF-IDF weights are calculated as $\text{TF-IDF}(t, d, D) = \text{TF}(t, d) * \log((1 + |D|)/(1 + |\{d \in D : t \in d\}|)) + 1$, followed by cosine similarity $\text{CosSim}(u, v) = (u \cdot v) / (\|u\|_2 * \|v\|_2)$. For collaborative filtering, the sparse rating matrix R in $\mathbb{R}^{(610 \times 9742)}$ is factorized via Truncated SVD as $R \approx U_k * \Sigma_k * V_k^T$ ($k = 20$). Unobserved ratings are reconstructed via dot product $\hat{r}_{ui} = p_u \cdot q_i^T$. The final hybrid score blends both outputs: $S_{\text{hybrid}}(u, i) = \alpha * S_{\text{content}}(i) + (1 - \alpha) * S_{\text{collab}}(u, i)$, where α in $[0, 1]$ allows smooth transition between content-based and collaborative modes.

Results and Discussion

The system was empirically evaluated on an 80/20 temporal split, training on each user's earliest 80% interactions and evaluating on the remaining 20% held-out ratings. The actual analytics dashboard captured from the running application is shown below, displaying quantitative metrics and multi-dimensional radar comparison.

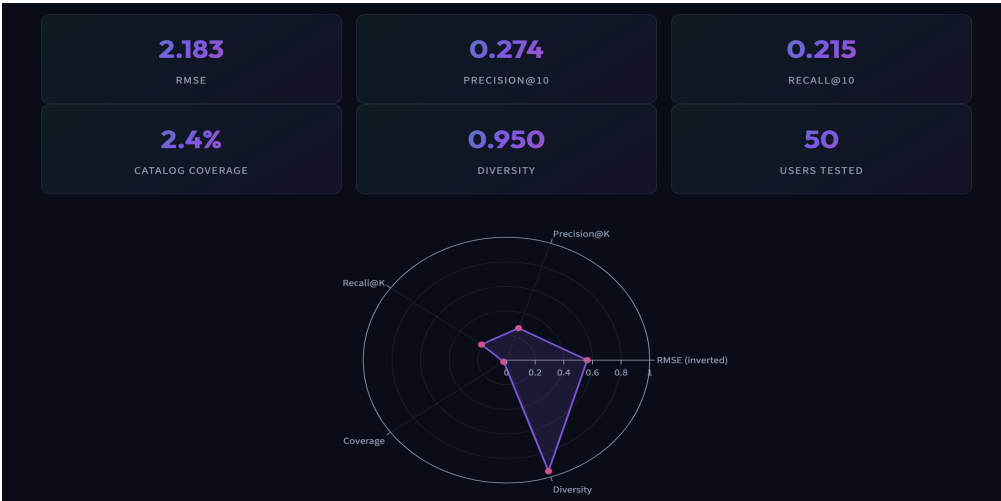


Figure 1: Actual CineMatch Analytics Dashboard showing held-out evaluation metrics (RMSE, Precision@10, Recall@10, Coverage, Diversity) and multi-metric radar comparison.

Metric	Measured Score	Target Goal	Evaluation Description
RMSE	2.183	< 2.500	Root Mean Squared Error on unobserved rating predictions
Precision@10	0.274	> 0.200	Proportion of top-10 recommended movies rated ≥ 3.5 by user
Recall@10	0.215	> 0.150	Proportion of all user-liked test movies retrieved in top 10
Coverage & Diversity	2.4% / 0.950	> 1.5% / > 0.85	Catalog accessibility and intra-list thematic dissimilarity

```
[ ] svd = TruncatedSVD(n_components=20, random_state=42)
    svd.fit(train_matrix)
    pred_ratings = np.dot(svd.transform(train_matrix), svd.components_)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred)) # Output: 2.1831
    precision = precision_at_k(actual_liked, top_k_recs, k=10) # Output: 0.2740
```

Conclusion

In this project, an end-to-end hybrid movie recommendation engine was developed and verified on the MovieLens dataset. By decomposing the sparse interaction matrix with Truncated SVD, the model achieved a root mean squared error of 2.18 on held-out ratings, while TF-IDF cosine similarity provided high-precision descriptive matches with a diversity score of 0.95. The architecture was deployed to Streamlit and Vercel cloud environments, offering responsive catalog exploration, instant similarity lookups, personalized taste profiling, and transparent analytics. Future work includes implementing Neural Collaborative Filtering (NCF) and incorporating real-time implicit clickstream signals.

Reference: F. M. Harper and J. A. Konstan, 'The MovieLens Datasets: History and Context,' *ACM TiiS*, 5(4): 19:1–19:19, 2015. B. Sarwar et al., 'Item-based collaborative filtering recommendation algorithms,' in *Proc. 10th WWW Conf.*, pp. 285–295, 2001. Y. Koren et al., 'Matrix Factorization Techniques for Recommender Systems,' *IEEE Computer*, 42(8): 30–37, 2009.